Consejería de Educación, Cultura y Deportes
IES VIRREY MORCILLO
AVDA. MENÉNDEZ PELAYO S/N
967 140 881 02003120.ies@edu.jccm.es

UT1 · FUNDAMENTOS DE KOTLIN

Manual técnico

Arquitectura, código y mantenimiento del proyecto EjerciciosKotlin

Proyecto	EjerciciosKotlin (IntelliJ IDEA, módulo Kotlin/JVM sin Gradle)
Repositorio	github.com/ccnl10/EjerciciosKotlin
Lenguaje	Kotlin 2.4 · JVM target 1.8 · JDK 21
Versión del documento	2.0 · octubre de 2026 · bloques 1 a 5
Destinatarios	Profesorado y alumnado que mantenga o amplíe el proyecto
Documentos relacionados	Manual de usuario · README.md · docs/api/index.html

Índice

1. Introducción 3
1.1 Cifras del proyecto 3
1.2 Entorno 3
2. Estructura del proyecto 4
2.1 Paquetes 4
3. Arquitectura 5
3.1 Navegación entre menús 6
3.2 Menús internos de los ejercicios 7
3.3 Convenciones de nombres 7
4. Descripción de los paquetes 8
4.1 Paquete raíz y Utilidades 8
4.2 Bloque 1 · Variables, tipos y operadores 8
4.3 Bloque 2 · Condicionales, rangos y bucles 8
4.4 Bloque 3 · Funciones y lambdas 8
4.5 Bloque 4 · Nulos y colecciones 9
4.6 Bloque 5 · Clases, herencia e interfaces 9
5. Modelo de clases del bloque 5 11
5.1 El ejemplo del zoo 11
5.2 El refugio (reto del bloque 5) 12
6. Decisiones de diseño 13
7. Sistema de ayuda integrado 14
7.1 Funciones de ayuda de cada menú 14
7.2 Añadir la ayuda a un menú nuevo 14
8. Documentación del código 15
8.1 Comentarios KDoc 15
8.2 Página HTML de la API 15
8.3 Publicarla en GitHub Pages 15
8.4 Generarla con Dokka 16
9. Compilación, distribución y control de versiones 17
9.1 Ejecutar desde IntelliJ 17
9.2 Generar un .jar ejecutable 17
9.3 Git y GitHub 17
10. Cómo ampliar el proyecto 18
11. Pruebas realizadas 19
12. Cambios de esta versión y observaciones 20
12.1 Cambios realizados al documentar 20
12.2 Observaciones 20

1. Introducción

Este documento describe cómo está construido **EjerciciosKotlin**, la aplicación de consola que reúne los ejercicios resueltos de la **UT1 · Fundamentos de Kotlin** del módulo Desarrollo de Interfaces (2º DAM). Explica la organización en paquetes, el recorrido de una ejecución, las decisiones de diseño, el modelo de clases del bloque 5, el sistema de ayuda y los pasos para documentar, compilar, distribuir y ampliar el proyecto.

La referencia de cada función y clase (parámetros, valor de retorno, propiedades y comportamiento) no se repite aquí: está en los comentarios **KDoc** del código y en la página `docs/api/index.html`, generada a partir de ellos.

1.1 Cifras del proyecto

Elemento	Cantidad
Paquetes	8 (Utilidades, Bloque1 … Bloque5, Bloque5.EjemploZooB5, Bloque5.EjerciciosB5) más el paquete raíz
Archivos .kt	36 (unas 4.300 líneas, comentarios incluidos)
Funciones de nivel superior	148 (públicas; las privadas no se cuentan)
Clases, interfaces y objects	39, con 65 miembros públicos o protegidos
Constantes y propiedades de nivel superior	41
Ejercicios ejecutables desde los menús	42 (6 + 7 + 8 + 7 + 7 + 7) más las 6 opciones del ejemplo del zoo
Menús con `?. Ayuda`	10 (principal, 5 de bloque y 4 submenús)
Menús internos de ejercicio	5 (agenda, lista de productos, tienda, refugio y la propia tienda del reto)

1.2 Entorno

Componente	Versión / configuración	Dónde se define
JDK	Microsoft OpenJDK 21 (ms-21)	`.idea/misc.xml`
Kotlin	languageVersion y apiVersion 2.4	`.idea/kotlinc.xml`
Destino JVM	jvmTarget 1.8	`.idea/kotlinc.xml`
Biblioteca	KotlinJavaRuntime	`.idea/libraries`
Carpeta de fuentes	`src` (raíz de fuentes del módulo)	`EjerciciosKotlin.iml`
Salida de compilación	`out/` (excluida de Git)	`.idea/misc.xml` y `.gitignore`
Control de versiones	Git, rama `master`, remoto en GitHub	`.git/config`

> **NOTA · Proyecto sin Gradle**
>
> El proyecto es un módulo IntelliJ «puro»: no tiene `build.gradle.kts`. Se compila y ejecuta desde el IDE. Eso lo hace muy sencillo para clase, pero las herramientas que dependen de Gradle (como el plugin de Dokka) requieren pasos adicionales, que se explican en el apartado 8.

2. Estructura del proyecto

ÁRBOL DE CARPETAS

```
EjerciciosKotlin/
├── README.md                       Presentación del proyecto en GitHub
├── docs/
│   ├── api/index.html              Documentación HTML de la API (generada desde KDoc)
│   ├── Manual_Tecnico_EjerciciosKotlin.docx / .pdf
│   └── Manual_Usuario_EjerciciosKotlin.docx / .pdf
├── EjerciciosKotlin.iml            Definición del módulo IntelliJ
├── .idea/                          Configuración del proyecto (JDK, Kotlin)
├── .gitignore
└── src/
    ├── Main.kt                     main(), menú principal y su ayuda
    ├── Utilidades/
    │   ├── Colores.kt              Colores ANSI, mostrarAviso(), mostrarError()
    │   └── Ayuda.kt                mostrarAyuda(): cuadro de ayuda común
    ├── Bloque1/  MenuB1.kt · EjerciciosB1.kt
    ├── Bloque2/  MenuB2.kt · EjerciciosB2.kt
    ├── Bloque3/
    │   ├── MenuB3.kt               Elige parte 1 o parte 2
    │   ├── MenuB3_1.kt · EjerciciosB3_1.kt · FuncionesB3_1.kt
    │   └── MenuB3_2.kt · EjerciciosB3_2.kt · FuncionesB3_2.kt
    ├── Bloque4/  MenuB4.kt · EjerciciosB4.kt · FuncionesB4.kt
    └── Bloque5/
        ├── MenuB5.kt               Elige el ejemplo del zoo o los ejercicios
        ├── EjemploZooB5/           Paquete Bloque5.EjemploZooB5
        │   ├── Animal.kt · Caninos.kt · Felinos.kt · Hipopotamo.kt
        │   ├── Mascota.kt · Veterinario.kt
        │   └── MenuZooB5.kt · OpcionesZooB5.kt
        └── EjerciciosB5/           Paquete Bloque5.EjerciciosB5
            ├── MenuEjerciciosB5.kt · EjerciciosB5.kt
            ├── Cachorro.kt · Pez.kt · Errores.kt · Instrumentos.kt
            ├── Aves.kt · Especies.kt
            └── AnimalesRefugio.kt · Refugio.kt
```

2.1 Paquetes

Paquete	Responsabilidad	Depende de
(raíz)	Punto de entrada y menú principal	Bloque1 … Bloque5, Utilidades
Utilidades	Colores de consola, avisos, errores y cuadro de ayuda. No conoce a ningún bloque.	—
Bloque1	Variables, tipos, operadores y plantillas	Utilidades (solo el menú)
Bloque2	Condicionales, rangos y bucles	Utilidades (solo el menú)
Bloque3	Funciones (parte 1) y lambdas (parte 2)	Utilidades (solo los menús)
Bloque4	Nulos y colecciones: listas, conjuntos y mapas	Utilidades
Bloque5	Menú del bloque 5	Los dos subpaquetes, Utilidades
Bloque5.EjemploZooB5	Jerarquía de animales del ejemplo de Java y su menú de demostración	Utilidades
Bloque5.EjerciciosB5	Ejercicios de clases, herencia, interfaces y clases especiales	Utilidades

Las dependencias van siempre en un solo sentido: la raíz conoce a los bloques, los bloques conocen a `Utilidades` y `Utilidades` no conoce a nadie. Ningún bloque depende de otro, de modo que cada uno puede modificarse o eliminarse sin afectar a los demás.

El bloque 5 es el único con **subpaquetes**. Cada carpeta declara su paquete completo en la primera línea (`package Bloque5.EjemploZooB5`), y `MenuB5.kt` importa los dos menús: `import Bloque5.EjemploZooB5.menuZooB5` e `import Bloque5.EjerciciosB5.menuEjerciciosB5`.

3. Arquitectura

El código se organiza en tres capas, más una de utilidades que usan todas:

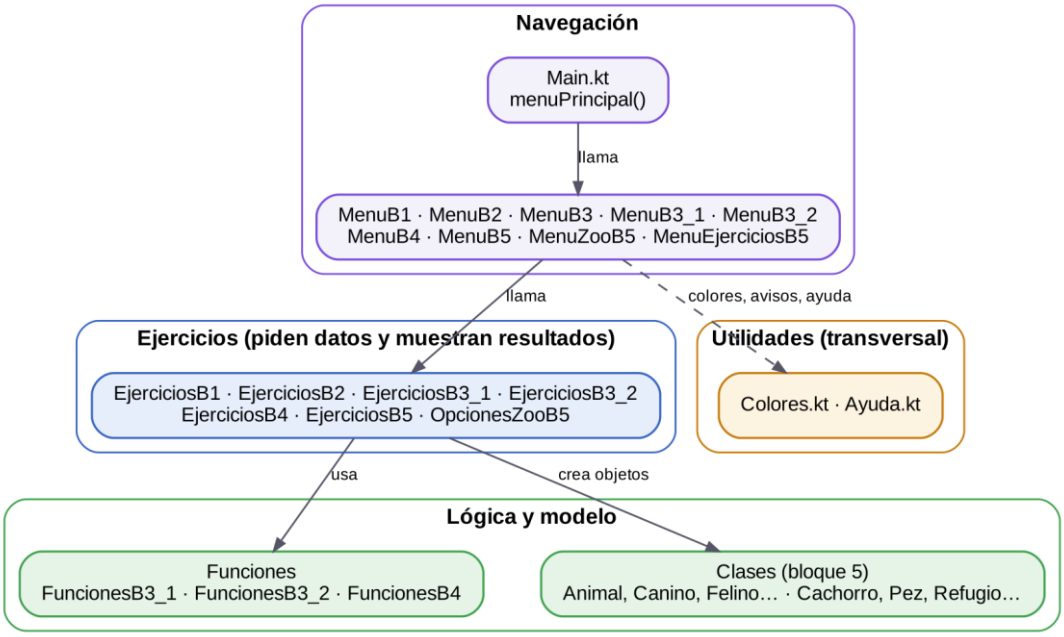


Figura 1. Capas del proyecto y sentido de las llamadas

Capa	Archivos	Qué hace	Qué no hace
Navegación	`Main.kt`, `Menu*.kt`	Pinta un menú, lee la opción y llama al ejercicio o submenú	No calcula nada ni conoce los datos de los ejercicios
Ejercicios	`Ejercicios*.kt`, `OpcionesZooB5.kt`	Pide los datos al usuario, llama a la lógica o crea objetos y muestra el resultado	Desde el bloque 3 no contiene cálculos: los delega
Lógica y modelo	`Funciones*.kt` y las clases del bloque 5	Funciones con parámetros y retorno; clases con sus propiedades y métodos	Casi nunca lee de teclado (excepciones: `leerNumeroEnRango`, `leerTexto`…)
Utilidades	`Colores.kt`, `Ayuda.kt`	Presentación común: colores, avisos, errores y ayuda	No depende de ningún bloque

> **NOTA · Bloques 1 y 2**
> En los bloques 1 y 2 todavía no se habían visto las funciones con parámetros, así que cada `bNEjercicioN()` contiene su propia lógica. La separación entre ejercicio y lógica empieza en el bloque 3, que es justamente el tema de ese bloque.

3.1 Navegación entre menús

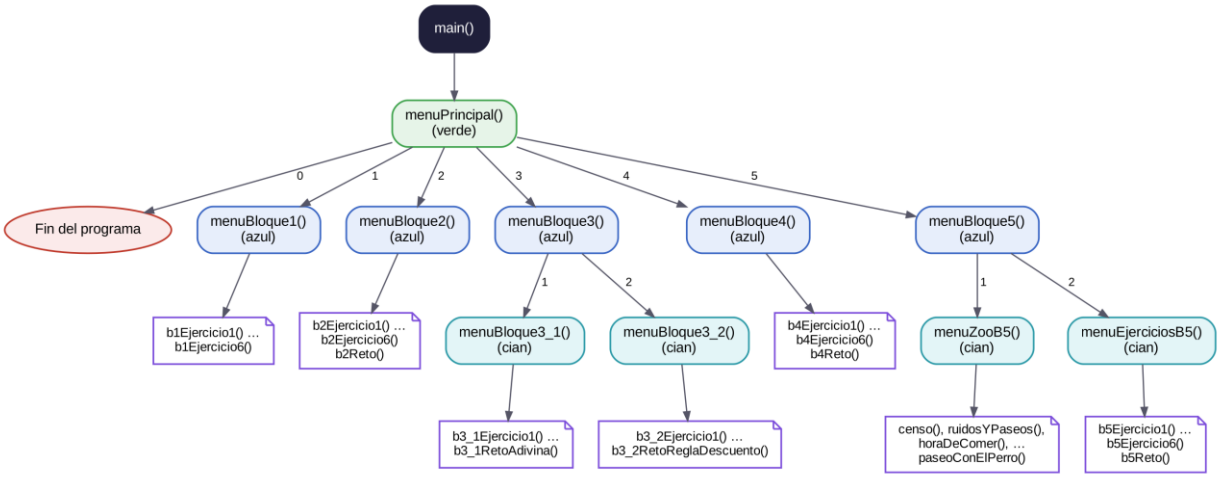


Figura 2. Árbol de menús

Todos los menús siguen el mismo patrón: un bucle `do-while` que se repite hasta que el usuario elige **0**. Para volver **no se llama** al menú anterior: basta con que el bucle termine. La función acaba y la ejecución continúa en el `when` del menú que la llamó, cuyo bucle vuelve a pintarlo.

PATRÓN COMÚN DE TODOS LOS MENÚS

```
fun menuBloque4() {
    var opcion: Int?
    var entrada: String?

    do {
        // 1. Pintar el menú (en color)
        print(AZUL)
        println("1. Entrada segura")
        ...
        println("0. Volver al menú principal")
        println("?. Ayuda")
        print("Elige una opción: ")
        print(RESET)

        // 2. Leer la opción como texto y como número
        entrada = readln().trim()
        opcion = entrada.toIntOrNull()

        // 3. "?" muestra la ayuda y vuelve a pintar el menú
        if (entrada == "?") {
            ayudaBloque4()
            continue
        }

        // 4. Ejecutar la opción elegida
        when (opcion) {
            1 -> b4Ejercicio1()
            ...
            0 -> mostrarAviso("Volviendo al menú principal...")
            else -> mostrarError("Opción no válida, inténtalo de nuevo.")
        }
    } while (opcion != 0)   // 5. Al elegir 0, el bucle termina y se vuelve
}
```

> **IMPORTANTE · No llamar al menú anterior para volver**
> Si la opción de volver llamase a `menuPrincipal()`, se abriría un menú nuevo encima del anterior (recursión). Tras navegar un rato, habría que elegir «Salir» varias veces y, en casos extremos, se produciría un `StackOverflowError`. Con el patrón del bucle solo hay un menú de cada nivel abierto en cada momento.

Con `?`, `continue` salta a la condición del `do-while`. Como `opcion` vale `null`, la condición `opcion != 0` se cumple y el menú se vuelve a pintar. Usar **0** para volver tiene una ventaja frente a usar el último número: al añadir un ejercicio no hay que cambiar la condición del `while`.

3.2 Menús internos de los ejercicios

Algunos ejercicios tienen su propio menú: la agenda (`b4Ejercicio3`), la lista de productos (`b4Ejercicio6`), la tienda (`b4Reto`) y el refugio (`b5Reto`). Son más sencillos que los de navegación: comparan la entrada como **texto** (`when (entrada) { "1" -> … }`), no usan colores ni ayuda, y salen con su última opción (`5`, `7` o `0` según el ejercicio). La tienda sale además al terminar la compra.

3.3 Convenciones de nombres

Elemento	Convención	Ejemplos
Paquetes	PascalCase, uno por bloque; subpaquetes con punto	`Bloque4`, `Bloque5.EjemploZooB5`
Archivos	PascalCase, según su papel	`MenuB2.kt`, `EjerciciosB3_1.kt`, `FuncionesB4.kt`
Menús	`menuBloqueN()`; submenús con nombre propio	`menuBloque3_2()`, `menuZooB5()`
Ejercicios	`bNEjercicioM()` y `bNReto…()`	`b2Ejercicio4()`, `b5Reto()`
Ayuda de cada menú	`ayuda…()`	`ayudaBloque3Parte1()`, `ayudaZooB5()`
Funciones de lógica	camelCase, empiezan por un verbo	`calcularMedia()`, `anadirAlCarrito()`
Clases e interfaces	PascalCase, sustantivo en singular	`Animal`, `Mascota`, `FichaAnimal`
Constantes	`const val` en UPPER_SNAKE_CASE	`NUMERO_SECRETO`, `HAMBRE_MAXIMA`

Los nombres con guion bajo (`b3_1Ejercicio1`) no siguen la convención camelCase de Kotlin, por lo que IntelliJ puede mostrar un aviso de estilo. Se mantienen porque identifican de forma inequívoca el bloque y la parte; el aviso no afecta a la compilación. La convención oficial también recomienda paquetes en minúscula (`bloque1`); si se quiere cambiar, IntelliJ lo hace en un paso con **Refactor → Rename**.

4. Descripción de los paquetes

Resumen de lo que contiene cada archivo. El detalle de cada función y clase está en `docs/api/index.html`.

4.1 Paquete raíz y Utilidades

Archivo	Declaraciones	Descripción
Main.kt	`main()`, `menuPrincipal()`, `ayudaMenuPrincipal()`	Arranca el programa y muestra el menú principal en verde
Colores.kt	`ROJO`, `VERDE`, `AMARILLO`, `AZUL`, `CIAN`, `MORADO`, `RESET`, `mostrarAviso()`, `mostrarError()`	Códigos ANSI y mensajes de aviso (amarillo) y de error (rojo)
Ayuda.kt	`mostrarAyuda(titulo, lineas)`	Cuadro de ayuda común a todos los menús; espera a que el usuario pulse Intro

4.2 Bloque 1 · Variables, tipos y operadores

Función	Ejercicio	Conceptos
`b1Ejercicio1()`	Preséntate	`val`, plantillas `$x` y `${expr}`
`b1Ejercicio2()`	¿val o var?	Mutabilidad y tipos básicos
`b1Ejercicio3()`	Encuentra los errores	Reasignación, tipos numéricos, `toLong()`, división entera
`b1Ejercicio4()`	Conversor de temperatura	`readln()`, `toDoubleOrNull()`, `format()`
`b1Ejercicio5()`	Calculadora de precio	Operadores aritméticos y `%.2f`
`b1Ejercicio6()`	Recibo del presupuesto	`const val`, operador Elvis `?:`, alineación `%-21s%10.2f`

4.3 Bloque 2 · Condicionales, rangos y bucles

Función	Ejercicio	Conceptos
`b2Ejercicio1()`	Gastos de envío	`if` como expresión, constantes, `Locale`
`b2Ejercicio2()`	De nota a calificación	`when` con rangos
`b2Ejercicio3()`	Tabla de multiplicar	`for`, `..` y `downTo`
`b2Ejercicio4()`	Predice la salida	`step`, rangos vacíos, orden en `when`, `do-while`, `break`/`continue`
`b2Ejercicio5()`	Mes del año	`when` con `else`
`b2Ejercicio6()`	Mes y estación	`!in`, `when` sin sujeto, retorno anticipado
`b2Reto()`	Adivina el número	`while`, `break`, `continue`, contadores

4.4 Bloque 3 · Funciones y lambdas

Ejercicio	Funciones que usa
`b3_1Ejercicio1()` Saludos	`mostrarBienvenida`, `saludar`, `saludarEnIdioma`
`b3_1Ejercicio2()` Conversor	`celsiusAFahrenheit`, `kmAMillas`, `esPar`, `maximoDeTres`, `formatear`
`b3_1Ejercicio3()` Usuarios	`crearNombreUsuario`
`b3_1Ejercicio4()` Guardas	`calcularMedia`, `mostrarResultado`

Ejercicio	Funciones que usa
`b3_1Ejercicio5()` Errores	`duplicar`, `incrementar`, `total`, `esMayorDeEdad`, `mostrarPrecio`
`b3_1Ejercicio6()` Calculadora	`calcularSubtotal`, `calcularImporteDescuento`, `calcularPrecioConDescuento`, `calcularImporteImpuesto`, `calcularPrecioFinal` y las de presentación
`b3_1RetoMesEstacion()`	`leerMes`, `nombreMes`, `estacionDe`
`b3_1RetoAdivina()`	`mostrarReglasJuego`, `jugarAdivinarNumero`, `mostrarPista`, `mostrarResultadoJuego`
`b3_2Ejercicio1()` De función a lambda	`cuadrado`, `esPar`, `nombreCompleto`, `imprimirSeparador`
`b3_2Ejercicio2()` Tipos de función	Solo lambdas locales
`b3_2Ejercicio3()` Calculadora	`calcular`
`b3_2Ejercicio4()` Analizador	`analizarTexto`
`b3_2Ejercicio5()` Tu propio repeat	`repetirAccion`
`b3_2Ejercicio6()` Predice la salida	`transformar`
`b3_2RetoReglaDescuento()`	`sinDescuento`, `blackFriday`, `descuentoEstudiante`, `aplicarRegla`, `combinarReglas`, `mostrarPrecioConRegla`

Las dos partes del bloque 3 están en el mismo paquete, así que las funciones de una parte se pueden usar desde la otra sin `import`. Por eso `esPar` y `formatear` no se repiten en `FuncionesB3_2.kt`: declararlas otra vez con la misma firma daría el error **Conflicting overloads**.

4.5 Bloque 4 · Nulos y colecciones

Ejercicio	Funciones de FuncionesB4.kt que usa	Conceptos
`b4Ejercicio1()` Entrada segura	`leerEnteroOPorDefecto`	`toIntOrNull()`, Elvis `?:`
`b4Ejercicio2()` Predice la salida	—	`?.`, `?:` y `null` en plantillas
`b4Ejercicio3()` Agenda	`anadirContacto`, `mostrarContactos`, `buscarContacto`, `eliminarContacto`	`MutableList`, `any`, `filter`, menú interno
`b4Ejercicio4()` Notas	`mediaNotas`, `notaMasAlta`, `notaMasBaja`, `contarAprobados`, `suspensosOrdenados`, `subirUnPunto`, `hayAlgunDiez`	`average`, `maxOrNull`, `count`, `filter`, `map`, `any`
`b4Ejercicio5()` Visitantes	`usuariosUnicos`, `visitasPorUsuario`	`Set`, `groupingBy().eachCount()`
`b4Ejercicio6()` Productos	`mostrarProductos`, `anadirOActualizarProducto`, `eliminarProducto`, `mostrarPrecioProducto`, `mostrarMasBaratos`, `mostrarResumenCatalogo`	`MutableMap`, `maxByOrNull`, `sumOf`
`b4Reto()` Tienda con stock	`mostrarCatalogoTienda`, `anadirAlCarrito`, `calcularSubtotalCarrito`, `mostrarCarrito`, `mostrarTotales`, `mostrarTicket`	Dos mapas y una lista; sin `!!`

`FuncionesB4.kt` incluye además funciones de lectura reutilizables (`leerTexto`, `leerDecimalPositivo`, `leerEnteroOPorDefecto`) y su propio formateador, `formatearB4()`, que escribe los importes con coma decimal usando `ESPANOL_B4`.

4.6 Bloque 5 · Clases, herencia e interfaces

`MenuB5.kt` solo ofrece dos caminos: **1. Ejemplo del zoo** (`menuZooB5()`) y **2. Ejercicios del bloque 5** (`menuEjerciciosB5()`).

Bloque5.EjemploZooB5

Archivo	Declaraciones	Papel
Animal.kt	`HAMBRE_MINIMA`, `HAMBRE_MAXIMA`, `HAMBRE_INICIAL`, `UMBRAL_HAMBRE`, `abstract class Animal`	Raíz de la jerarquía
Caninos.kt	`abstract class Canino`, `Lobo`, `Perro`	La familia de los caninos en un archivo
Felinos.kt	`abstract class Felino`, `Gato`	La familia de los felinos
Hipopotamo.kt	`Hipopotamo`	Hereda directamente de `Animal`
Mascota.kt	`interface Mascota`	Comportamiento de `Perro` y `Gato`
Veterinario.kt	`Veterinario`	`vacunar(animal: Animal)`: polimorfismo por parámetro
MenuZooB5.kt	`menuZooB5()`, `ayudaZooB5()`	Solo navegación
OpcionesZooB5.kt	`crearZoo()`, `censo()`, `ruidosYPaseos()`, `horaDeComer()`, `visitaVeterinario()`, `horaDeJugar()`, `paseoConElPerro()`	Una función por opción del menú

Bloque5.EjerciciosB5

Ejercicio	Archivo y clases	Conceptos
`b5Ejercicio1()` Tu primera clase	Cachorro.kt · `Cachorro`	Constructor primario, `init`, getter personalizado
`b5Ejercicio2()` Setter con validación	Pez.kt · `Pez`	Setter con `field`, constructor secundario
`b5Ejercicio3()` Encuentra los errores	Errores.kt · `Insecto`, `Reptil`, `Mamifero`, `Planta`, `Roedor` y sus subclases	`open`, `override`, `abstract`
`b5Ejercicio4()` Instrumentos	Instrumentos.kt · `Instrumento`, `Guitarra`, `Piano`, `Bateria`	Clase abstracta, polimorfismo, `is`
`b5Ejercicio5()` Aves	Aves.kt · `Ave`, `Nadador`, `Volador`, `Pato`, `Pinguino`, `Aguila`	Interfaces con propiedades y métodos con cuerpo
`b5Ejercicio6()` Fichas del refugio	Especies.kt · `Especie`, `FichaAnimal`, `consejo()`	`enum class`, `data class`, `copy`, desestructuración
`b5Reto()` Refugio de animales	AnimalesRefugio.kt y Refugio.kt · `Adoptable`, `Animal`, `Perro`, `Gato`, `Adopcion`, `Refugio`, `Animal.etiqueta()`	`companion object`, `object`, función de extensión, `as?`

5. Modelo de clases del bloque 5

5.1 El ejemplo del zoo

El paquete `Bloque5.EjemploZooB5` reproduce en Kotlin la jerarquía del ejemplo de herencia de Java que se trabajó en clase:

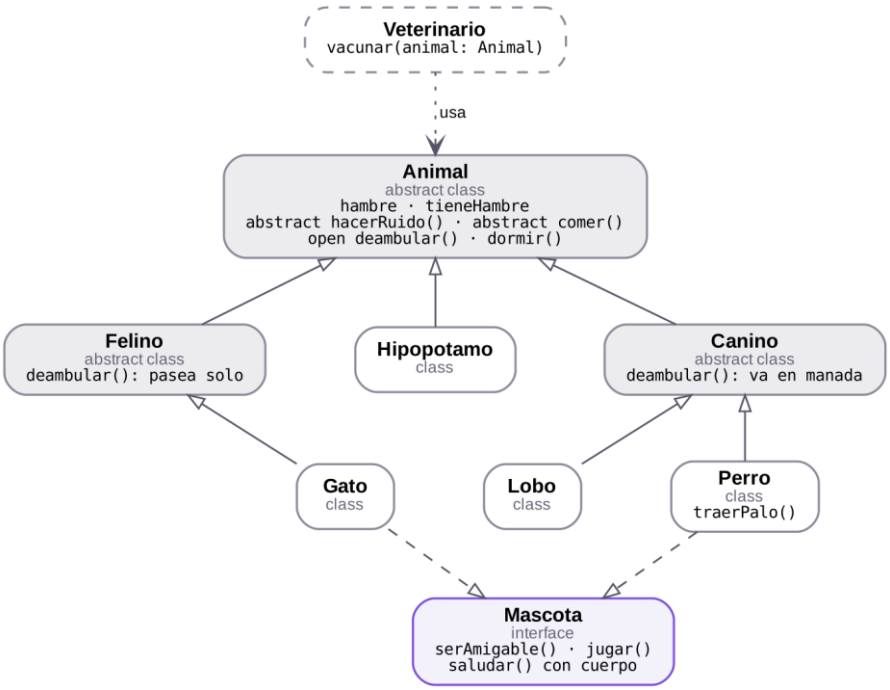


Figura 3. Jerarquía del zoo. Flecha continua: hereda. Flecha discontinua: implementa. Fondo gris: clase abstracta.

Decisión	Motivo
`Animal`, `Canino` y `Felino` son abstractas	No tiene sentido crear «un animal» o «un canino» sin más; solo sus subclases concretas
`hacerRuido()` y `comer()` son abstractos	Cada animal los hace a su manera, así que la primera clase concreta está obligada a implementarlos
`deambular()` es `open`	`Canino` y `Felino` lo sobrescriben; `Hipopotamo` usa el de `Animal`
`dormir()` está cerrado	Todos los animales duermen igual; nadie puede cambiarlo
`hambre` tiene el setter `protected` con validación	Desde fuera solo se lee; solo `comer()` de cada subclase la modifica, y nunca sale del rango 0-10
`tieneHambre` es una propiedad calculada	Se deduce de `hambre` cada vez que se lee; no se guarda
`Mascota` es una interfaz	`Perro` y `Gato` ya heredan de `Canino` y `Felino`, y en Kotlin solo se hereda de una clase
`Mascota` declara `val nombre`	Lo necesita `saludar()`; `Perro` y `Gato` no lo reescriben porque ya lo heredan de `Animal`

`OpcionesZooB5.kt` enseña cada concepto con los mismos cuatro animales, que `crearZoo()` crea al entrar en el menú y se mantienen mientras se está en él: por eso, al elegir dos veces «Hora de comer», la segunda vez ya no tienen hambre.

5.2 El refugio (reto del bloque 5)

- `Animal` (de `Bloque5.EjerciciosB5`) asigna a cada animal un `id` automático con un contador privado dentro de su `companion object`.
- `Refugio` es un `object`: hay un único refugio en todo el programa. `inicializar()` registra los animales de ejemplo solo la primera vez, para que no se dupliquen al volver a entrar en el reto.
- `Animal.etiqueta()` es una función de extensión que usa `as? Adoptable` para mostrar la cuota sin fallar si un animal no es adoptable.

> **IMPORTANTE · Nombres repetidos en dos paquetes**
>
> `Animal`, `Perro` y `Gato` existen en `Bloque5.EjemploZooB5` y en `Bloque5.EjerciciosB5`. Compila porque están en paquetes distintos, pero si un archivo importa los dos paquetes con `*`, el compilador no sabrá cuál usar (**Ambiguity**). En ese caso, importa la clase concreta o usa un alias: `import Bloque5.EjemploZooB5.Perro as PerroZoo`.

6. Decisiones de diseño

Un paquete por bloque

Permite que cada bloque tenga sus propios nombres sin conflictos. Por ejemplo, `NUMERO_SECRETO` existe en `Bloque2` y en `Bloque3` porque son ejercicios independientes.

Lectura de datos tolerante a errores

Se usa `toIntOrNull()` y `toDoubleOrNull()` en lugar de `toInt()` y `toDouble()`, que lanzan una excepción si el texto no es un número. Los decimales se leen con `replace(',', '.')` para aceptar `21,5` y `21.5`. Según el ejercicio, un valor no válido se sustituye por un valor por defecto con el operador Elvis, muestra un mensaje o se vuelve a pedir (`leerNumeroEnRango`, `leerDecimalPositivo`). Desde el bloque 4 no se usa `!!` en ningún sitio.

Constantes en lugar de números mágicos

Los valores fijos de los enunciados (gastos de envío, número secreto, IVA, límites del hambre o de la temperatura…) se declaran con `const val` al principio del archivo que los usa.

Formato de los decimales

Los bloques 2, 3 y 4 muestran los importes con coma decimal pasando el `Locale` español a `format()` (`formatear()` en el bloque 3, `formatearB4()` en el 4). Así el resultado no depende del idioma del sistema operativo. El bloque 1 usa `format()` sin `Locale`, por lo que sus importes salen con coma en un Windows en español y con punto en uno en inglés.

Colores ANSI

Los colores se aplican imprimiendo códigos de escape ANSI (`\u001B[32m`). Cada menú imprime su color al principio y `RESET` antes de leer la opción, para que ni la entrada del usuario ni la salida de los ejercicios hereden el color. Los avisos y errores usan `mostrarAviso()` y `mostrarError()`, que añaden el `RESET`.

Color	Constante	Uso
Verde	`VERDE`	Menú principal
Azul	`AZUL`	Menús de bloque (1 a 5)
Cian	`CIAN`	Submenús: bloque 3 (partes 1 y 2) y bloque 5 (zoo y ejercicios)
Amarillo	`AMARILLO`	Avisos y ayuda
Rojo	`ROJO`	Errores
Morado	`MORADO`	Sin uso por ahora

Reutilización

`leerNumeroEnRango()` se escribió para el reto del mes y se reutiliza en el juego de adivinar. `mostrarAyuda()` da a todos los menús la misma presentación de la ayuda. En el bloque 5, `Veterinario.vacunar()` y las opciones del zoo trabajan con el tipo `Animal`, así que sirven para cualquier animal presente o futuro.

7. Sistema de ayuda integrado

Cada menú de navegación acepta `?` como entrada y muestra una ayuda contextual: una línea por opción, explicando qué hace cada ejercicio. La presentación (marco, color amarillo y pausa hasta pulsar Intro) está centralizada en `Utilidades/Ayuda.kt`; cada menú solo aporta el texto.

Utilidades/Ayuda.kt

```
fun mostrarAyuda(titulo: String, lineas: List<String>) {
    print(AMARILLO)
    println()
    println("---------------- AYUDA  -----------------")
    println(titulo)
    println("----------------------------------------")
    for (linea in lineas) {
        println(linea)
    }
    println("----------------------------------------")
    println("Pulsa Intro para volver al menú...")
    print(RESET)
    readln()
}
```

7.1 Funciones de ayuda de cada menú

Menú	Función de ayuda
Principal	`ayudaMenuPrincipal()`
Bloques 1, 2, 3 y 4	`ayudaBloque1()` … `ayudaBloque4()`
Bloque 3 · parte 1 y parte 2	`ayudaBloque3Parte1()`, `ayudaBloque3Parte2()`
Bloque 5	`Bloque5.ayudaBloque5()`
Ejemplo del zoo	`ayudaZooB5()`
Ejercicios del bloque 5	`Bloque5.EjerciciosB5.ayudaBloque5()`

7.2 Añadir la ayuda a un menú nuevo

1. Añade la línea `println("?. Ayuda")` al final de las opciones del menú.
2. Lee la entrada como texto (`entrada = readln().trim()`) y después conviértela: `opcion = entrada.toIntOrNull()`.
3. Antes del `when`, añade `if (entrada == "?") { ayudaMiMenu(); continue }`.
4. Escribe `ayudaMiMenu()` en el mismo archivo del menú, llamando a `mostrarAyuda()` con el título y una lista de líneas.

8. Documentación del código

8.1 Comentarios KDoc

Todas las funciones, constantes, clases, interfaces y miembros públicos tienen un comentario **KDoc**, el formato estándar de documentación de Kotlin. IntelliJ lo muestra al pasar el ratón por encima de una llamada (o con **Ctrl + Q**).

EJEMPLO DE KDOC (Veterinario.kt)

```
/**
 * Vacuna a un animal y hace que haga su ruido.
 *
 * El parámetro es de tipo [Animal], así que sirve para cualquier subclase:
 * se ejecuta el [Animal.hacerRuido] del objeto que llega (polimorfismo).
 *
 * @param animal animal que se vacuna.
 */
fun vacunar(animal: Animal) {
```

Elemento	Uso
Primera frase	Resumen de lo que hace la declaración
`@param nombre`	Describe un parámetro de una función
`@property nombre`	Describe una propiedad declarada en el constructor de una clase
`@return`	Describe el valor devuelto
`[nombre]`	Enlace a otra declaración; IntelliJ, Dokka y la página de la API lo convierten en un vínculo
Comillas invertidas	Texto en fuente de código, por ejemplo un nombre de variable

Los comentarios `//` del código (explicaciones de los enunciados, predicciones, errores corregidos) explican el **porqué** del código; el KDoc describe **qué hace** cada declaración para quien la usa.

8.2 Página HTML de la API

`docs/api/index.html` reúne todos los KDoc organizados por paquete y archivo. Para cada clase muestra su firma, de quién hereda, sus subclases conocidas y sus miembros; tiene buscador, enlaces entre declaraciones y modo claro u oscuro. Se abre con doble clic en cualquier navegador y no necesita conexión.

La página se genera con un script de Python que lee directamente los comentarios de `src`. Si se cambia el código, hay que volver a generarla para que no se quede desactualizada.

8.3 Publicarla en GitHub Pages

1. Sube la carpeta `docs` al repositorio.
2. En GitHub, entra en **Settings** → **Pages**.
3. En **Source** elige **Deploy from a branch**, la rama `master` y la carpeta `/docs`, y pulsa **Save**.
4. Al cabo de un minuto, la API está en `https://ccnl10.github.io/EjerciciosKotlin/api/`.

8.4 Generarla con Dokka

Dokka es la herramienta oficial de JetBrains que genera documentación HTML a partir de los KDoc, igual que Javadoc en Java. Como el proyecto no usa Gradle, la forma más directa es la versión de línea de comandos:

1. Descarga de Maven Central los archivos `dokka-cli`, `dokka-base`, `analysis-kotlin-descriptors`, `kotlinx-html-jvm` y `freemarker`, todos de la misma versión de Dokka, y guárdalos en una carpeta `dokka`.
2. Abre una consola en la carpeta del proyecto y ejecuta el comando del recuadro.
3. Abre `docs/dokka/index.html` en el navegador.

CONSOLA DE WINDOWS

```
java -jar dokka/dokka-cli-1.9.20.jar ^
  -pluginsClasspath "dokka/dokka-base-1.9.20.jar;dokka/analysis-kotlin-descriptors-
1.9.20.jar;dokka/kotlinx-html-jvm-0.8.0.jar;dokka/freemarker-2.3.31.jar" ^
  -sourceSet "-src src -jdkVersion 21" ^
  -outputDir docs/dokka
```

En proyectos reales lo habitual es usar Gradle con el plugin `id("org.jetbrains.dokka")` y ejecutar la tarea `dokkaHtml`. Es el mismo sistema de dependencias que se usará con Jetpack Compose.

9. Compilación, distribución y control de versiones

9.1 Ejecutar desde IntelliJ

Abre `src/Main.kt` y pulsa el ▶ junto a `fun main()`. Comprueba que el desplegable de configuraciones de ejecución muestra `MainKt`: si muestra otra clase (por ejemplo, de un archivo que antes tenía su propio `main`), el menú no aparecerá.

9.2 Generar un .jar ejecutable

1. **File → Project Structure → Artifacts → + → JAR → From modules with dependencies**.
2. En **Main Class** elige `MainKt` y pulsa **OK**. IntelliJ incluye la biblioteca de Kotlin dentro del .jar.
3. **Build → Build Artifacts → EjerciciosKotlin:jar → Build**.
4. El archivo se genera en `out/artifacts/EjerciciosKotlin_jar/EjerciciosKotlin.jar`.

> **EJECUCIÓN EN WINDOWS**
>
> ```
> chcp 65001
> java -jar EjerciciosKotlin.jar
> ```

`chcp 65001` cambia la consola a UTF-8 para que las tildes, la ñ y los símbolos (€, °, π) se vean bien. Para ver los colores conviene usar **Windows Terminal**; el cmd clásico de Windows 10 puede mostrar los códigos ANSI como texto (`←[32m`).

El `.jar` no debe subirse al repositorio (la carpeta `out/` está en `.gitignore`). Si se quiere distribuir, lo adecuado es adjuntarlo a una **Release** de GitHub.

9.3 Git y GitHub

El proyecto está en `https://github.com/ccnl10/EjerciciosKotlin`, rama `master`. El historial tiene un commit por cada ampliación importante (estructura del bloque 3, colores y ayuda, bloque 4, bloque 5).

Qué	Se sube	Motivo
`src/`, `README.md`, `docs/`	Sí	Código y documentación
`EjerciciosKotlin.iml`, `.idea/` (salvo `workspace.xml`)	Sí	Permite abrir el proyecto con la misma configuración
`out/`, `.kotlin`	No	Se generan al compilar (ya están en `.gitignore`)
`.idea/workspace.xml`	Mejor no	Guarda el estado personal del IDE (pestañas abiertas, ejecuciones…)

> **SUBIR LOS CAMBIOS**
>
> ```
> git add src docs README.md
> git commit -m "Documentación: manuales, API y README"
> git push
> ```

10. Cómo ampliar el proyecto

Pasos para añadir un bloque nuevo (por ejemplo, el bloque 6) siguiendo la misma estructura:

1. Crea el paquete: clic derecho sobre `src` → **New** → **Package** → `Bloque6`.
2. Crea `EjerciciosB6.kt` con `package Bloque6` y una función por ejercicio: `b6Ejercicio1()`, `b6Ejercicio2()`…
3. Separa la lógica reutilizable en `FuncionesB6.kt` y, si hay clases, dales su propio archivo.
4. Crea `MenuB6.kt` copiando `MenuB4.kt`: cambia el título, las opciones, las llamadas y la función de ayuda. La opción de volver sigue siendo **0**, así que no hay que tocar la condición del `while`.
5. En `Main.kt`, añade `import Bloque6.menuBloque6`, la línea `6. Bloque 6 · …` en el menú, la rama `6 -> menuBloque6()` en el `when` y una línea en `ayudaMenuPrincipal()`.
6. Documenta las declaraciones nuevas con KDoc y vuelve a generar `docs/api/index.html`.

> **CONSEJO · Bloques con varias partes**
> Si un bloque tiene partes muy distintas, sigue el modelo del bloque 5: un `MenuBN.kt` que solo elige la parte, y un subpaquete por parte (`BloqueN.ParteA`, `BloqueN.ParteB`) con su propio menú en cian. Así cada parte puede repetir nombres de clases sin conflictos.

11. Pruebas realizadas

El proyecto se ha compilado completo con `kotlinc` 2.x y se han ejecutado los ejercicios con los datos de los enunciados. Resultados:

Ejercicio	Entrada	Resultado esperado	OK
B1 · 4 Conversor	21,5	70.7 °F (53.5 °F con la fórmula incorrecta)	✓
B1 · 6 Recibo	Teclado · 25 · 4 · 10 · Intro	Impuesto 21 %; TOTAL 108,90	✓
B2 · 1 Envío	42,5	Envío 4,99 €; a pagar 47,49 €	✓
B2 · Reto	15, 4, 8, 7	15 no cuenta; mayor; menor; acierto en 3 intentos	✓
B3.1 · 6 Calculadora	Datos del enunciado	Precio final 190,67 €	✓
B3.2 · 3 Calculadora	12 · 4 · *	12.0 * 4.0 = 48.0	✓
B3.2 · Reto	—	50 € → 35,00 / 45,00 / 31,50; 15 € → 10,50 / 15,00 / 10,50	✓
B4 · 1 Entrada segura	25 · abc	Edad 25; entradas 1 (valor por defecto)	✓
B4 · 3 Agenda	Laura, Pablo, vacío; buscar «lau»; borrar Pablo	Rechaza el vacío; encuentra Laura; queda 1 contacto	✓
B4 · 4 Notas	—	Media 6,21; 5 de 7 aprobados; suspensos [3.0, 4.0]	✓
B4 · 6 Productos	ratón 10 · webcam 45,5 · borrar monitor	Valor total 120,40 €; más caro webcam	✓
B4 · Reto	gorra ×2 · sudadera · zapatos	Sin stock; agotado; no encontrado; total 11,50 €	✓
B5 · 2 Pez	—	35 rechazado; 26,5 aceptado; Dory(40) se queda en 24	✓
B5 · 6 Fichas	—	copy con edad 4; ¿iguales? true; 110 min	✓
B5 · Reto	1 · 2 misi/Laura · 2 Kira · 3	Adopción de Misi; Kira no existe; ids 1-3	✓
Zoo · 3 Hora de comer	Dos veces seguidas	1.ª: comen todos; 2.ª: no tienen hambre	✓
Zoo · 6 Paseo	—	Toby trae el palo; Misi no (as? da null)	✓
Menús	? en cada menú	Muestra la ayuda y vuelve a pintar el menú	✓
Menús	Texto o número fuera de rango	Opción no válida (en rojo)	✓
Navegación	Ida y vuelta por todos los menús	Un solo 0 en el menú principal cierra el programa	✓

12. Cambios de esta versión y observaciones

12.1 Cambios realizados al documentar

- Añadidos comentarios KDoc a `Main.kt`, `Colores.kt` y los bloques 1, 2 y 3, que no los tenían. Los bloques 4 y 5 ya estaban documentados. Solo se han añadido comentarios: el código no cambia.
- Añadidos `docs/api/index.html`, este manual, el manual de usuario y un `README.md` nuevo.

12.2 Observaciones

- En `MenuEjerciciosB5.kt`, la opción **0** dice «Volver al menú principal», aunque vuelve al menú del bloque 5 (el aviso sí dice «Volviendo al menú del bloque 5...»). Su título, «MENÚ - EJERCICIOS DEL BLOQUE 5», es el mismo que el de `MenuB5.kt`, y su KDoc dice que vuelve a `menuPrincipal`.
- Hay dos funciones `ayudaBloque5()`: una en `Bloque5` y otra en `Bloque5.EjerciciosB5`. Compila porque están en paquetes distintos, pero conviene renombrar la segunda (por ejemplo, `ayudaEjerciciosB5()`) y cambiar en su texto «Volver al menú principal» por «Volver al menú del bloque 5».
- Las constantes `NUMERO_SECRETO`, `NUMERO_MINIMO`, `NUMERO_MAXIMO`, `MAX_INTENTOS` y `ESPANOL` están duplicadas en `Bloque2` y `Bloque3`. Es intencionado (cada bloque es autónomo), pero si se cambian en uno no cambian en el otro.
- El formato decimal del bloque 1 depende del idioma del sistema (ver apartado 6).
- La constante `MORADO` está definida pero no se usa.
- Los mensajes del bloque 1 están en inglés, como sus enunciados; los del resto de bloques, en español.
- Para la funcionalidad no hay pruebas automáticas (carpeta `test`); las pruebas del apartado 11 se han hecho ejecutando el programa.